

Week 4 →

Policy Evaluation and Control :-

- 1) Policy evaluation is the task of determining the value function for the given specific policy.
 - 2) Policy Control is the task of finding a policy to obtain as much reward as possible.
- ↑
ultimate goal
of Reinforcement learning.

Dynamic Programming :-

- ✗ DP algorithms use the Bellman equations to define iterative algorithms for both policy evaluation and control.

1) Policy Evaluation :

$$\pi \rightarrow V_{\pi}$$

- ✗ Here the job is to determine how good the policy is!
- ✗ Here we determine V_{π} from π .

$$V_{\pi}(s) = E_{\pi} [G_t \mid s_t = s]$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{k+t+1}$$

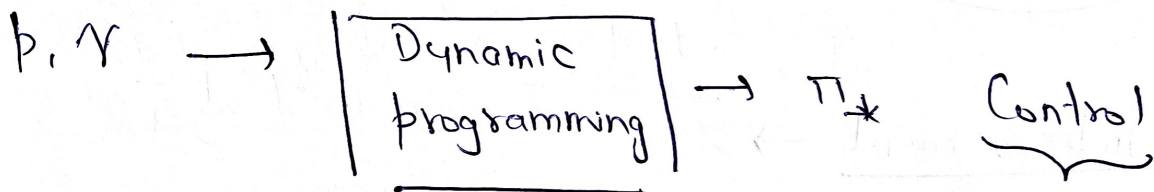
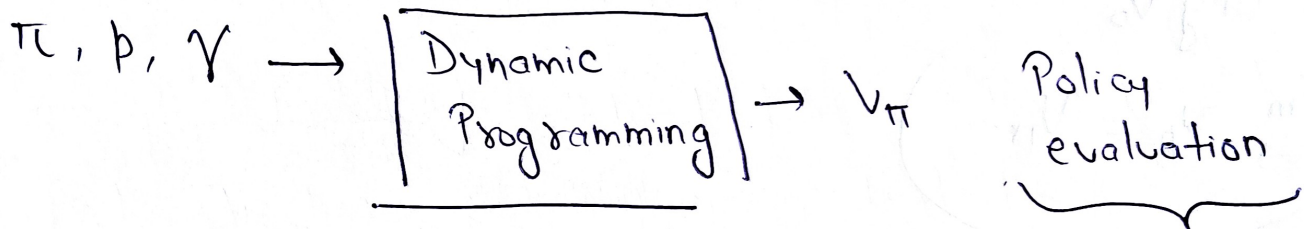
2) Control :

- ✗ It is the task of improving a policy.

∅ problem of policy evaluation reduces to solving the system of linear equation.

Assumptions :-

Suppose we have access to the Dynamics of the environment i.e p



Iterative policy iteration $\rightarrow$

∅ Dynamic programming algorithms are obtained by turning the bellman equations into update rule.

∅ Here we will use the iterative policy iteration algorithm for estimating state values under a given policy.

$$V_{k+1}^{(s)} \leftarrow \sum_s \pi(a|s) \sum_{s'} \sum_a p(s', a | s, a) [r + \gamma V_k^{(s')}] \quad \left. \vphantom{\sum_s \sum_{s'} \sum_a} \right\} \begin{array}{l} \text{update} \\ \text{policy} \\ \text{obtained} \\ \text{from} \\ \text{bellman} \\ \text{eqn} \end{array}$$

(This is the update rule for our value functions and we will keep on doing this

until $V_{k+1}^{(s)} = V_k^{(s)}, \forall s \in \mathcal{S}$)

which is V_π

(because V_π is the unique soln of bellman eqn)

for any V_0

$$\lim_{k \rightarrow \infty} V_k = V_\pi$$

Policy Improvement $\rightarrow$

Recall that

$$\pi_*(s) = \underset{a}{\operatorname{argmax}} \sum_{s'} \sum_a p(s', a | s, a) [r + \gamma V_*(s')]$$

Greedy action

(instead of the optimal value function, we select an action which is greedy with respect to the value function V_π of an arbitrary policy π) ?

Algorithm (Iterative policy Evaluation)
for estimating $V \approx V_\pi$

(14)

- 1) Input π , the policy to be evaluated
- 2) $V \leftarrow \vec{0}$, $V' \leftarrow \vec{0}$
- 3) Loop:
- 4) $\Delta \leftarrow 0$
- 5) Loop for each $s \in \mathcal{S}$
- 6) $V'(s) \leftarrow \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$
- 7) $\Delta \leftarrow \max(\Delta, |V'(s) - V(s)|)$
- 8) $V \leftarrow V'$
- 9) Until $\Delta < \theta$
- 10) Output $V \approx V_\pi$

$$\pi'(s) \leq \arg \max_a \sum_{s'} \sum_{r} p(s', r | s, a) [r + \gamma V_\pi(s')]$$

this new policy

must be different from π . If this greedyfication doesn't ~~change~~ change π , then π was already greedy with respect to its own value function.

This is just another way of saying that V_π obeys the bellman's optimality equation.

$$T_{\pi}(s) = \arg \max_a \sum_{s'} \sum_{a'} p(s', a' | s, a) [r + \gamma V_{\pi}(s')], \forall s \in \mathcal{S}$$

V_{π} obeys bellman optimality eqn $\Rightarrow \pi$ is optimal

∅ The new policy is a strict improvement over π unless π is already optimal.

Policy Improvement Theorem $\rightarrow$

$$Q_{\pi'}(s, \pi'(s)) \geq Q_{\pi}(s, \pi(s)), \forall s \in \mathcal{S}$$

$$\rightarrow \boxed{\pi' \geq \pi}$$

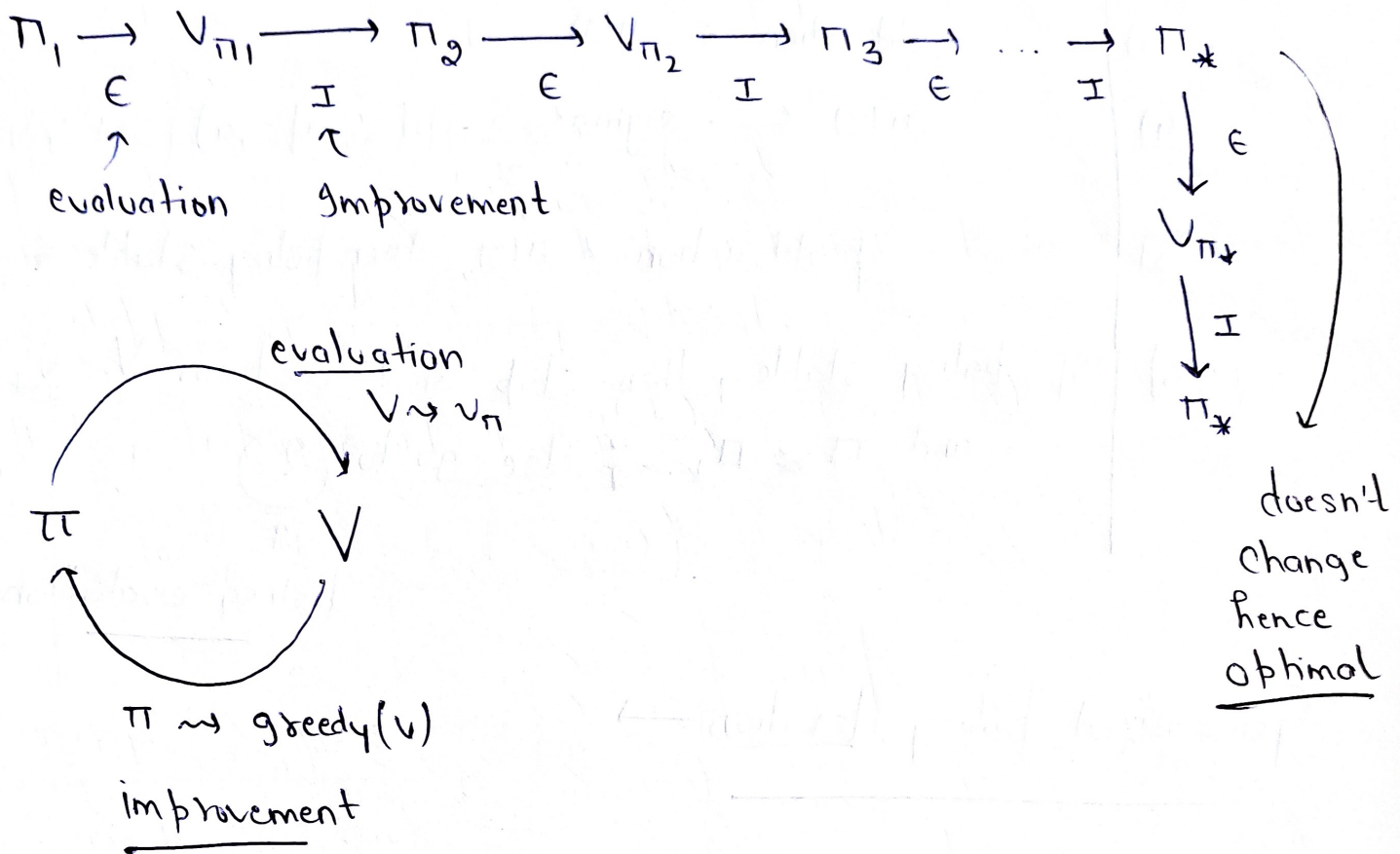
$$Q_{\pi'}(s, \pi'(s)) > Q_{\pi}(s, \pi(s)), \text{ for at least one } s \in \mathcal{S}$$

$$\rightarrow \boxed{\pi' > \pi}$$

- ∅ Thus the policy improvement theorem tells us that a greedy policy is a strict improvement unless the original policy was already optimal.
- ∅ Use the value function under a given policy, to produce a strictly better policy.

Policy iteration $\rightarrow$

(15)



Algorithm (Policy iteration)

1) Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily, $\forall s \in \mathcal{S}$

2) Policy Evaluation

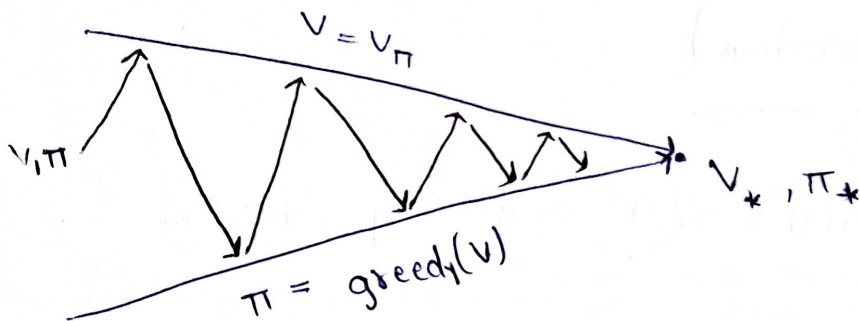
(iterative policy Evaluation)

3) Policy Improvement

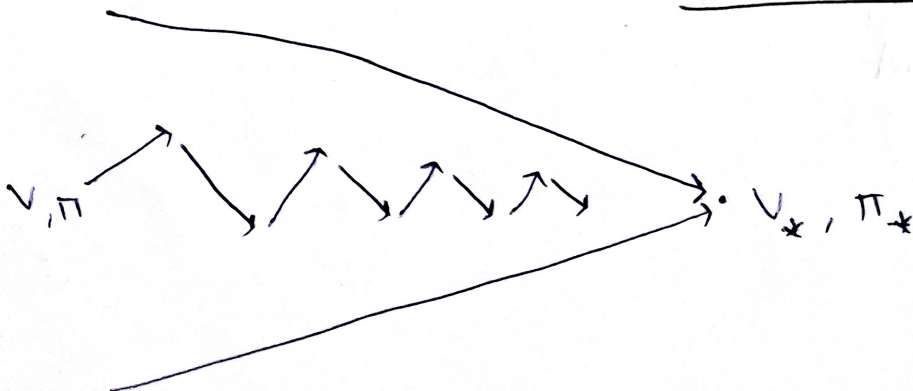
- 1) policy stable $\leftarrow$ true
 - 2) for each $s \in \mathcal{S}$:
 - 3) old action $\leftarrow \pi(s)$
 - 4) $\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s', a} p(s', a | s, a) (r + \gamma V(s'))$
 - 5) if old action $\neq \pi(s)$, then policy stable $\leftarrow$ false
 - 6) if policy stable, then stop and return $V \approx V_*$ and $\pi \approx \pi_*$; else go to. (2)
- ↑
policy evolution

Generalized policy iteration $\rightarrow$

Policy iteration



Generalized policy iteration



⇒ Value iteration, an important special case of Generalized policy iteration! (16)

Value iteration → In this, we will sweep over all the states and greedify with respect to the current value function. However we do not run the policy evaluation to completion.

perform
∅ we just sweep over all the states!

Evaluation step!

∅ After that we greedify again.

Algorithm (Value iteration) →

- 1) Loop:
- 2) $\Delta \leftarrow 0$
- 3) Loop for each $s \in \mathcal{S}$:
- 4) $v \leftarrow V(s)$
- 5) $V(s) \leftarrow \max_a \sum_{s', a'} p(s', a' | s, a) [\gamma + r V(s')]$
- 6) $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
- 7) Until $\Delta < \theta$
- 8) Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \operatorname{argmax}_a \sum_{s', a'} p(s', a' | s, a) \cdot [\gamma + r V(s')]$$

- ∅ we can write this as a update rule which applies directly to the state value function.
- ∅ The update does not reference to any specific policy, hence the name Value iteration.
- ∅ Instead of updating the value according to a fixed policy, we update using the action that maximizes the current value estimate.
- ∅ Value iteration still converges to the V^* in the limit.

Avoiding full sweeps →

Synchronous DP :-

- ∅ Methods that perform ~~systematic~~ sweeps (like full sweeps) are called synchronous.
- ∅ This can be problematic in case state space is large.

Asynchronous DP :-

- ∅ Here we update the values of states in any order, they do not perform systematic sweeps.
- ∅ This can take less time.
- ∅ Some states can be updated more than others however for the algorithm to converge each and every state should be updated.

Efficiency of Dynamic Programming →

(17)

⇒ A sampling alternative for Policy evaluation:-

- ∅ Iterative policy evaluation is the dynamic programming solution to the prediction or policy evaluation problem.
- ∅ Sample base alternative: The value of each state can be treated as totally independent estimation problem

$$V_n(s) \doteq E_n[G_t | s_t = s]$$

- 1) Gather a large number of returns under π and take their average. This will gradually converge to state value, called the monte carlo method.
- 2) However, we need large number of returns from each state.
- 3) Each return depends on many random actions, Selected by π , as well as many random state transitions due to dynamics of the MDP.
- 4) However, the key insight of DP is that we do not have to treat the evaluation of each state as a separate problem. We can use the other value estimates we have already worked. This process of using the value estimates of successor state to improve our current value estimate is known as bootstrapping.

↑

much more efficient than Monte Carlo method that estimates each value independently.

→ Policy iteration computes optimal policies, brute force search is a possible alternative.

- 1) This method simply ~~evaluates~~ evaluates every possible deterministic policy one at a time. we then pick the one with the highest value.

$\pi(s_1)$	$\pi(s_2)$		$\pi(s_{ S })$
a_3	a_1		a_2
a_4	a_2		a_3
			
a_2	a_7		a_3

however the number
of deterministic
policies can be huge.

$$= |A|^{|S|}$$

Then,
— efficiency of DP →

- 1) Policy iteration :- Polynomial time in $|S|$ and $|A|$

Brute force search :- $|A|^{|S|}$